SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I ***R.Jagan-192321102*** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

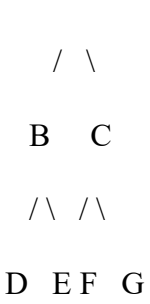
Signature of the student: ***R.Jagan***

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



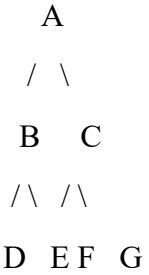
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

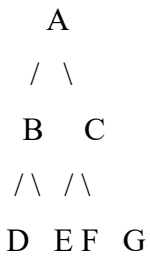
RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Answers:

1.Breadth-First Search (BFS) on the Tree

Graph: A is the root; A → B, C; B → D, E; C → F, G (left-to-right order of children).



Algorithm (BFS uses a FIFO Queue)

- Put the start node in the queue.
- Remove (dequeue) the node at the **front**, mark it visited, and add all its unvisited children to the **rear** of the queue (left to right).
- Repeat until the queue is empty or the goal is found. BFS explores level by level: Level 0 → Level 1 → Level 2.

Dry-run table

Iteration	**Queue (before removing front)**	**Visited Node**	**Queue after iteration (children added)**
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G] (D has no children)
5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[ ] (empty – search ends)

(i) BFS traversal order

A → B → C → D → E → F → G

(ii) Queue contents after each iteration

After iteration 1: [B, C] | 2: [C, D, E] | 3: [D, E, F, G] | 4: [E, F, G] | 5: [F, G] | 6: [G] | 7: [ ]

Remarks

- Nodes are visited in order of increasing depth (A; then B, C; then D, E, F, G), so BFS is complete and finds the shallowest goal; it is optimal when all step costs are equal.
- Time complexity O(b^d) = 7 nodes here; space complexity O(b^d) because the whole frontier is stored (maximum queue length here = 4).

2.Depth-First Search (DFS) on the Same Tree

Algorithm (DFS uses a LIFO Stack): push the start node; pop the **top** node, mark it visited, then push its children so that the **left child is on top** (i.e., push the right child first, then the left). Continue until the stack is empty. DFS goes as deep as possible along one branch before backtracking.Dry-run table (top of stack written on the left)

Iteration	**Stack (before pop)**	**Visited Node**	**Stack after iteration**
1	[A]	A	[B, C] (push C, then B)
2	[B, C]	B	[D, E, C] (push E, then D)
3	[D, E, C]	D	[E, C] (leaf – backtrack)
4	[E, C]	E	[C] (leaf – backtrack to A's other child)
5	[C]	C	[F, G] (push G, then F)
6	[F, G]	F	[G]
7	[G]	G	[ ] (empty – search ends)

(i) DFS traversal order

A → B → D → E → C → F → G

(ii) Comparison with BFS and remarks

Feature	**BFS**	**DFS**
Data structure	Queue (FIFO)	Stack (LIFO) / recursion
Order on this tree	A B C D E F G	A B D E C F G
Max frontier size here	4 nodes (D, E, F, G)	3 nodes (e.g., D, E, C)
Memory	O(b^d) – large	O(b·m) – small (m = max depth)
Complete / Optimal	Complete; optimal for equal step cost	Not complete in infinite/cyclic graphs; not optimal

DFS needs less memory and is suitable when the goal is deep, but it may go down a wrong long path; BFS guarantees the shallowest solution.

3.BFS on the 8-Puzzle

Blank tile is written as _ (0). A move slides the blank **Up, Down, Left or Right**. Expansion order used: **Up, Down, Left, Right** (moves that would go off the board are skipped). Repeated states are not added again (explored-set check) and the goal test is done when a state is removed from the queue.

Initial State	**Goal State**
1 3 6 5 _ 2 4 7 8	1 2 3 4 5 6 7 8 _

Check: the number of inversions in the initial state is 6 (even), so the puzzle is solvable.

State labels used in the table

Label	**State**	**Obtained from**	**Move of blank**
S0	1 3 6 5 _ 2 4 7 8	Start	-
S1	1 _ 6 5 3 2 4 7 8	S0	Up
S2	1 3 6 5 7 2 4 _ 8	S0	Down
S3	1 3 6 _ 5 2 4 7 8	S0	Left
S4	1 3 6 5 2 _ 4 7 8	S0	Right
S5	_ 1 6	S1	Left

Label	State	Obtained from	Move of blank
	5 3 2 4 7 8		
S6	1 6 _ 5 3 2 4 7 8	S1	Right
S7	1 3 6 5 7 2 _ 4 8	S2	Left
S8	1 3 6 5 7 2 4 8 _	S2	Right
S9	_ 3 6 1 5 2 4 7 8	S3	Up
S10	1 3 6 4 5 2 _ 7 8	S3	Down
S11	1 3 _ 5 2 6 4 7 8	S4	Up
S12	1 3 6 5 2 8 4 7 _	S4	Down

Dry-run table (first 5 iterations)

Iteration	Current State (expanded)	Queue Before Expansion	Queue After Expansion
1	S0	[S0]	[S1, S2, S3, S4]

Iteration	Current State (expanded)	Queue Before Expansion	Queue After Expansion
			added: S1(Up), S2(Down), S3(Left), S4(Right)
2	S1	[S1, S2, S3, S4]	[S2, S3, S4, S5, S6] added: S5(Left), S6(Right)
3	S2	[S2, S3, S4, S5, S6]	[S3, S4, S5, S6, S7, S8] added: S7(Left), S8(Right)
4	S3	[S3, S4, S5, S6, S7, S8]	[S4, S5, S6, S7, S8, S9, S10] added: S9(Up), S10(Down)
5	S4	[S4, S5, S6, S7, S8, S9, S10]	[S5, S6, S7, S8, S9, S10, S11, S12] added: S11(Up), S12(Down)

Iteration 1 expands the initial state S0 (blank in the centre: 4 legal moves). Iterations 2–5 expand the four depth-1 states S1–S4, each of which has 2 new successors (the move back to the parent is rejected as a repeated state). The queue therefore grows by one state per iteration at this stage.

(i) Which node is expanded in each iteration?

Iteration 1: S0 (initial); 2: S1; 3: S2; 4: S3; 5: S4 – i.e. nodes are expanded in the order they entered the queue (FIFO), first all depth-0 and depth-1 states, then depth 2, depth 3, … until the goal comes to the front of the queue. The goal is at depth 8, and it is S310 (the 311th state expanded).

(ii) Total number of states generated

Because the shortest solution is 8 moves long, the table cannot end after 5 iterations; BFS must continue until the goal is removed from the queue. With the conventions above, BFS **expands 311 states** and **generates 500 states** in total (including the initial state).

Depth	States generated at this depth	States expanded	Remarks
0	1	1	Initial state

Depth	States generated at this depth	States expanded	Remarks
1	4	4	
2	8	8	
3	8	8	
4	16	16	
5	32	32	
6	60	60	
7	72	72	
8	136	110	Goal found while expanding depth 8
9	163	0	Generated by depth-8 expansions
Total	**500**	**311**	

(The exact totals depend on the move order and on whether repeated states are checked; with a different convention the count differs slightly, but the solution depth of 8 and the path length are always the same.)

(iii) Complete solution path (8 moves)

Step	Move of blank	Resulting state
0	Initial	1 3 6 5 _ 2 4 7 8
1	Right	1 3 6 5 2 _ 4 7 8
2	Up	1 3 _ 5 2 6 4 7 8

Step	Move of blank	Resulting state
3	Left	1 _ 3 5 2 6 4 7 8
4	Down	1 2 3 5 _ 6 4 7 8
5	Left	1 2 3 _ 5 6 4 7 8
6	Down	1 2 3 4 5 6 _ 7 8
7	Right	1 2 3 4 5 6 7 _ 8
8	Right	1 2 3 4 5 6 7 8 _

Move sequence: Right → Up → Left → Down → Left → Down → Right → Right. Path cost = **8**; the goal state is reached at depth 8.

(iv) Why BFS always finds the shortest solution in an unweighted 8-puzzle

- Every move has the **same cost (1)**, so path cost = number of moves = depth of the node.
- BFS expands nodes in **increasing order of depth** (FIFO queue): all nodes of depth d are expanded before any node of depth d+1.
- Therefore the first time the goal is reached, no shallower path to it can exist – otherwise it would have been found earlier. Hence BFS is **complete and optimal** here.
- **Drawback:** the number of states grows exponentially (about 3 new states per node: 1, 4, 8, 8, 16, 32, 60, 72, 136, …) so memory is large; heuristic search such as A* with Manhattan distance is much more efficient for deeper puzzles.